

Tutorial Sheet 3 - Linked List

Q1. Define a Linked List. How does it differ from an array in terms of memory allocation, insertion, and deletion? Draw the memory representation (diagram) of a singly linked list storing the elements $10 \rightarrow 20 \rightarrow 30$.

Q2. Explain the following types of linked lists with neat diagrams:

- a) Singly (Linear) Linked List
- b) Doubly Linked List
- c) Circular Linked List
- d) Header Linked List

Q3. Write an algorithm to perform the following operations on a singly linked list:

- a) Insert a node at the beginning.
- b) Insert a node at the end.
- c) Delete a node from the middle (by value).
- d) Traverse and print the linked list.

Q4. Consider a circular linked list with nodes storing student roll numbers.

- a) Write a pseudocode to traverse the list starting from any node and stop when the same node is reached again.
- b) Modify your code to insert a new student node after a given roll number.

Q5. Linked lists are widely used in real-world applications. Answer the following:

- 1) How are linked lists used in dynamic memory management (like malloc/free in C)?
- 2) Which type of linked list is most suitable for implementing a web browser's forward/backward navigation and why?
- 3) Implement a polynomial addition program using linked lists (each node stores coefficient and power).

Q6. Suppose you want to implement a music playlist app where:

- Songs can be added at the end.
- Users can move to the next or previous song.
- The playlist should repeat automatically after the last song.

Which type of linked list would you choose (singly, doubly, circular, or header)?
Justify your choice with a diagram.